

# Anàlisi de Costos Computacionals Asimptòtics

## Pràctica 1

Serafí Nebot Ginard

**Resum**—Aquesta memòria presenta el desenvolupament d’una aplicació Java que permet visualitzar i comparar el cost computacional asimptòtic de quatre algorismes amb complexitats computacionals  $O(n)$ ,  $O(n \cdot \log n)$ ,  $O(n^2)$  i  $O(n^3)$ . L’aplicació segueix estrictament el patró arquitectònic Model-Vista-Controlador (MVC), garantint una separació neta entre la lògica de negoci, la interfície gràfica i les dades. Es presenten els fonaments teòrics de l’anàlisi asimptòtic, el disseny del programari, la identificació de la instrucció crítica de cada algorisme, el càlcul de la constant multiplicativa i la predicció de temps d’execució per a mides d’entrada arbitràries.

### I. INTRODUCCIÓ

L’ANÀLISI d’algorismes és una de les disciplines fonamentals de la informàtica. Permet avaluar l’eficiència d’un algorisme en funció de la mida de l’entrada, independentment del maquinari o del llenguatge de programació emprat. El concepte central és el cost asimptòtic, que descriu com creix el temps d’execució quan la mida de l’entrada  $n$  tendeix a infinit.

Quan es dissenya un algorisme, sovint existeixen múltiples estratègies per resoldre el mateix problema. L’anàlisi asimptòtic proporciona una eina objectiva per comparar-les: un algorisme amb cost  $O(n \log n)$  serà sempre més eficient que un amb cost  $O(n^2)$  per a entrades prou grans, independentment de les constants implicades. Aquesta observació, que pot semblar abstracta, té conseqüències pràctiques immediates: la diferència entre un algorisme lineal i un cúbic pot ser la diferència entre una execució de mil·lisegons i una de dies.

L’objectiu d’aquesta pràctica és doble. D’una banda, es proposa implementar i mesurar experimentalment el temps d’execució de quatre algorismes amb costos asimptòtics coneguts: lineal, quasilineal, quadràtic i cúbic. D’altra banda, l’aplicació s’ha de construir seguint el patró Model-Vista-Controlador (MVC) amb comunicació basada en el patró per esdeveniments, un principi de disseny que desacobla completament les responsabilitats de cada capa del programari.

El resultat és una aplicació d’escriptori Java Swing que permet a l’usuari seleccionar rangs de mida d’entrada, executar algorismes de forma individual o conjunta, visualitzar els resultats en una gràfica en temps real (amb escala lineal o logarítmica), consultar les constants multiplicatives mitjançant fer previsions de temps per a mides d’entrada no mesurades.

La memòria s’estructura de la següent manera: la Secció II descriu l’entorn de programació; la Secció III exposa els fonaments teòrics; la Secció IV detalla el disseny i la implementació de la pràctica; la Secció V analitza el càlcul del cost asimptòtic per la instrucció crítica; la Secció VI presenta els resultats; i la Secció VII tanca amb les conclusions.

### II. ENTORN DE PROGRAMACIÓ

L’aplicació s’ha desenvolupat íntegrament en **Java**, un llenguatge orientat a objectes que ofereix portabilitat, tipatge

estàtic i un ecosistema madur de biblioteques. L’entorn concret és el següent:

- **Llenguatge:** Java 16 (compilació amb `source` i `target` 16).
- **JDK:** OpenJDK 23 (Valhalla), compatible cap enrere amb Java 16.
- **IDE:** IntelliJ IDEA, que proporciona eines d’anàlisi de codi, refactorització automatitzada i integració amb Maven.
- **Sistema de construcció:** Apache Maven 3.x, amb un fitxer `pom.xml` que defineix el `groupId` `com.serafinebot.pl`, la codificació UTF-8 i les versions del compilador.
- **Interfície gràfica:** Java Swing (paquets `javax.swing` i `java.awt`), sense cap biblioteca externa de gràfiques.
- **Control de versions:** Git.

La decisió d’utilitzar exclusivament Java Swing —sense biblioteques de tercers com JFreeChart— respon a un requisit de la pràctica i obliga a implementar manualment el renderitzat de la gràfica amb l’API `Graphics2D`, incloent-hi eixos, graella, sèries de dades i llegenda.

El fitxer `pom.xml` segueix una configuració mínima, ja que el projecte no té cap dependència externa:

```

1 <properties>
2   <maven.compiler.source>16</maven.compiler.source>
3   <maven.compiler.target>16</maven.compiler.target>
4   <project.build.sourceEncoding>
5     UTF-8
6   </project.build.sourceEncoding>
7 </properties>

```

Listing 1. Configuració Maven del projecte.

Aquesta absència de dependències externes és un aspecte rellevant del projecte: tota la funcionalitat —des del renderitzat de gràfiques fins a la concurrència— s’implementa amb les eines estàndard del JDK.

### III. FONAMENTS TEÒRICS

#### A. Anàlisi Asimptòtic de Costos

L’anàlisi asimptòtic permet classificar algorismes segons el seu comportament a llarg termini, és a dir, com creix el seu

temps d'execució en funció de la mida de l'entrada [3]. Es defineixen les notacions següents:

- $O(f(n))$  (cota superior asimptòtica): existeixen constants  $c > 0$  i  $n_0$  tals que  $T(n) \leq c \cdot f(n)$  per a tot  $n \geq n_0$ .
- $\Omega(f(n))$  (cota inferior): existeixen constants  $c > 0$  i  $n_0$  tals que  $T(n) \geq c \cdot f(n)$  per a tot  $n \geq n_0$ .
- $\Theta(f(n))$  (cota ajustada):  $T(n)$  és alhora  $O(f(n))$  i  $\Omega(f(n))$ .

El temps d'execució real d'un algorisme es pot modelar com:

$$T(n) = c \cdot f(n) \quad (1)$$

on  $f(n)$  és la funció de cost teòric i  $c$  és la *constant multiplicativa*, que depèn del maquinari, el compilador i altres factors constants. La notació asimptòtica extreu aquesta constant per a centrar-se només en l'ordre de creixement.

### B. Ordres de Creixement

Les quatre funcions de cost que s'analitzen en aquesta pràctica presenten comportaments de creixement dràsticament diferents. La Taula I il·lustra aquesta diferència amb valors numèrics concrets.

Taula I  
COMPARACIÓ D'ORDRES DE CREIXEMENT PER A DIFERENTS VALORS DE  $n$ .

| $n$    | $n$    | $n \log_2 n$ | $n^2$  | $n^3$     |
|--------|--------|--------------|--------|-----------|
| 10     | 10     | 33           | 100    | 1 000     |
| 100    | 100    | 664          | 10 000 | $10^6$    |
| 1 000  | 1 000  | 9 966        | $10^6$ | $10^9$    |
| 10 000 | 10 000 | 132 877      | $10^8$ | $10^{12}$ |

Com es pot observar, per a  $n = 10\,000$ , un algorisme cúbic realitza  $10^{12}$  operacions bàsiques, mentre que un de lineal només en fa  $10^4$ : una diferència de vuit ordres de magnitud. Si cada operació triga 1 nanosegon, l'algorisme lineal acabaria en 10 microsegons, mentre que el cúbic necessitaria aproximadament 17 minuts.

### C. Instrucció Crítica

La *instrucció crítica* és aquella operació dins l'algorisme que s'executa el major nombre de vegades i que, per tant, domina el cost total [7]. Identificar-la és clau per determinar la complexitat asimptòtica. En algorismes iteratius, la instrucció crítica és habitualment l'operació dins el bucle més intern. El nombre total d'execucions d'aquesta instrucció depèn del nombre d'iteracions de tots els bucles que l'envolten.

Per exemple, en un algorisme amb dos bucles imbricats, cadascun iterant  $n$  vegades, la instrucció crítica s'executa  $n \times n = n^2$  vegades, i per tant el cost asimptòtic és  $\Theta(n^2)$ .

### D. Patró Model-Vista-Controlador (MVC)

El patró MVC [1] divideix una aplicació en tres components amb responsabilitats ben definides:

- 1) **Model:** encapsula les dades i la lògica de negoci. Notifica als observadors quan el seu estat canvia, però

no coneix la interfície gràfica. En el nostre cas, emmagatzema les mesures d'execució, calcula constants multiplicatives i fa prediccions.

- 2) **Vista:** és responsable de la presentació visual. Renderitza les dades del model i captura les accions de l'usuari, però no coneix la lògica de negoci. En el nostre cas, dibuixa la gràfica, mostra la taula de resultats i ofereix els controls.
- 3) **Controlador:** actua com a intermediari. Reb events semàntics de la vista, invoca operacions sobre el model i actualitza la vista amb resultats. En el nostre cas, gestiona els fils d'execució dels algorismes.

Un principi fonamental del MVC ben implementat es que **la vista mai no ha d'exposar els seus components visuals** (botons, camps de text, etc.) al Controlador ni a cap altra capa. Tota comunicació ha de ser a través d'events semàntics i mètodes que operen amb dades. Aquesta restricció és crucial perquè, si el controlador accedís directament als botons o camps de text de la vista, qualsevol canvi a la interfície gràfica obligaria a modificar també el controlador, trencant el principi de separació de responsabilitats.

En implementacions ingènues del MVC, és comú veure patrons com `view.getButton().addActionListener(...)`, on el controlador obté directament un `JButton` de la vista. Això viola el patró perquè crea un acoblament directe entre el controlador i la representació visual concreta. En aquesta implementació, s'ha evitat completament aquest anti-patró.

### E. Patró per Esdeveniments (Observer)

El patró Observer [2] estableix una relació un-a-molts entre objectes, de manera que quan un objecte (el *subjecte*) canvia d'estat, tots els seus observadors són notificats automàticament. En el context d'aquesta pràctica, s'utilitza en dues direccions:

- 1) **Model → Vista:** El model defineix una interfície `ModelListener` amb el mètode `modelChanged()`. Quan s'afegeix una mesura o s'esborren dades, el model invoca aquest mètode, i la vista actualitza la taula i la gràfica.
- 2) **Vista → Controlador:** La vista defineix una interfície `ViewListener` amb set mètodes semàntics (`onStartAlgorithm`, `onStopAlgorithm`, `onStartAll`, `onStopAll`, `onPredict`, `onClear`, `onLogScaleChanged`). El controlador implementa aquesta interfície i reb events ja processats, mai components Swing.

Aquesta doble aplicació del patró Observer garanteix un desacoblament complet: el model no sap qui l'observa, i el controlador no sap com es la interfície gràfica. Es podria substituir tota la vista Swing per una interfície web o de terminal sense modificar ni una sola línia del model o del controlador.

### F. Concurrencia i el Fil EDT

En Java Swing, totes les operacions sobre components visuals s'han de realitzar des del fil de despatx d'events (*Event*

*Dispatch Thread*, EDT) [6]. Si un algorisme lent s'executa directament al fil EDT, la interfície es bloquejaria fins que acabes, impeding qualsevol interacció amb l'usuari.

La solució estàndard és *SwingWorker*, una classe que permet executar tasques llargues en un fil de fons i publicar resultats parcials al fil EDT de forma segura. El cicle de vida d'un *SwingWorker* és:

- 1) `doInBackground()`: s'executa en un fil de fons. Aquí s'executa l'algorisme i es crida `publish()` per enviar resultats intermedis.
- 2) `process(List<V>)`: s'invoca al fil EDT amb els resultats publicats. Aquí s'afegeixen les mesures al model.
- 3) `done()`: s'invoca al fil EDT quan la tasca acaba o es cancel·la. Aquí s'actualitza l'estat visual del botó.

La cancel·lació es cooperativa: `SwingWorker.cancel(true)` envia una interrupció al fil de fons, que ha de comprovar periòdicament `Thread.currentThread().isInterrupted()` per aturar-se netament.

#### IV. DISSENY I IMPLEMENTACIÓ DE LA PRÀCTICA

##### A. Estructura del Projecte

El projecte Maven s'organitza en tres paquets dins `com.serafinebot.pl1`:

```
src/main/java/com/serafinebot/pl1/
Main.java
model/
  AlgorithmType.java
  Measurement.java
  Model.java
view/
  ViewListener.java
  ControlPanel.java
  GraphPanel.java
  ResultsPanel.java
  MainView.java
controller/
  Controller.java
```

Cada paquet correspon exactament a una capa del patró MVC, amb el punt d'entrada `Main.java` a l'arrel. L'ordre d'instanciació segueix la seqüència clàssica:

```
1 Model model = new Model();
2 MainView view = new MainView(model);
3 new Controller(model, view);
4 view.setVisible(true);
```

Listing 2. Punt d'entrada de l'aplicació (`Main.java`).

El model es crea primer perquè no depen de res; la vista rep el model per poder registrar-se com a observadora dels seus canvis; i el controlador rep ambdós i es registra com a *ViewListener* de la vista. Tota la inicialització es realitza dins `SwingUtilities.invokeLater()` per garantir que es fa al fil EDT.

##### B. Capa del Model

La capa del model consta de tres classes, cadascuna amb una responsabilitat clara i ben delimitada.

1) *AlgorithmType* (Enum): Defineix els quatre algorismes com a constants d'un enum. Cada constant implementa dos mètodes abstractes:

- `execute(long n)`: executa l'algorisme amb mida  $n$  i retorna un resultat numèricament dependent de l'entrada per evitar que el compilador JIT l'optimitzi o l'elimini.
- `theoreticalCost(long n)`: retorna el valor numèric de  $f(n)$ .

A més, cada algorisme té associat un nom visible, una notació matemàtica i un color per a la gràfica. L'ús d'un enum amb mètodes abstractes és una aplicació del *Strategy pattern*: cada constant de l'enumeració encapsula el seu propi comportament.

Per exemple, l'algorisme quadràtic es defineix com:

```
1 QUADRATIC("O(n^2)", "Quadratic",
2         new Color(220, 120, 0)) {
3     public double theoreticalCost(long n) {
4         return (double) n * n;
5     }
6     public long execute(long n)
7         throws InterruptedException {
8         long sum = 0;
9         for (long i = 0; i < n; i++) {
10            if (Thread.currentThread()
11                .isInterrupted())
12                throw new InterruptedException();
13            for (long j = 0; j < n; j++) {
14                sum += i * j; // instrucció crítica
15            }
16        }
17        return sum;
18    }
19 };
```

Listing 3. Definició de l'algorisme  $O(n^2)$ .

Cal destacar dos aspectes: primer, la comprovació d'`isInterrupted()` al bucle extern (no al més intern, per no penalitzar el rendiment amb comprovacions massa freqüents); segon, el llançament d'`InterruptedException` que permet la cancel·lació cooperativa des del controlador.

2) *Measurement*: Classe immutable que emmagatzema el resultat d'una mesura: el tipus d'algorisme, la mida  $n$ , el temps en nanosegons i la constant multiplicativa, que es pre-càlcula al constructor:

$$c = \frac{T(n)}{f(n)} \quad (2)$$

on  $T(n)$  és el temps mesurat en nanosegons i  $f(n)$  és el cost teòric retornat per `theoreticalCost(n)`. El codi del constructor és:

```
1 public Measurement(AlgorithmType type,
2                 long n, long timeNanos) {
3     this.type = type;
4     this.n = n;
5     this.timeNanos = timeNanos;
6     double cost = type.theoreticalCost(n);
7     this.constant = cost > 0
8         ? timeNanos / cost : 0;
9 }
```

Listing 4. Constructor de *Measurement*.

Pre-càlculer  $c$  a cada mesura individual permet després obtenir la mitjana de forma directa, sense necessitat de recàlcul divisions repetidament.

3) *Model*: Classe central que manté un EnumMap de llistes de mesures. Els mètodes principals són:

- `addMeasurement(Measurement m)`: afegeix una mesura, ordena la llista per  $n$  creixent (per facilitar el dibuix de la gràfica) i notifica els observadors.
- `getAverageConstant(AlgorithmType t)`: retorna la mitjana aritmètica de les constants individuals, calculada amb l'API de *streams* de Java.
- `predict(AlgorithmType t, long n)`: càlcula el temps predit com  $\hat{T}(n) = \bar{c} \cdot f(n)$ , on  $\bar{c}$  es la constant mitjana.
- `clearAll()`: esborra totes les mesures i notifica.

Tots els mètodes que modifiquen o llegeixen l'estat són *synchronized*, ja que els fils de fons dels *SwingWorker* poden accedir al model concurrentment. El model implementa el patró Observer mitjançant la interfície *ModelListener*:

```
1 public interface ModelListener {
2     void modelChanged();
3 }
4
5 private void fireModelChanged() {
6     for (ModelListener l : listeners) {
7         l.modelChanged();
8     }
9 }
```

Listing 5. Patró Observer al model.

### C. Capa de la Vista

La vista es compon de quatre classes i una interfície. **Cap d'elles exposa components Swing a l'exterior.**

1) *ViewListener (Interfície)*: Defineix el contracte semantic entre la vista i el controlador:

```
1 public interface ViewListener {
2     void onStartAlgorithm(AlgorithmType type,
3         long[] nValues);
4     void onStopAlgorithm(AlgorithmType type);
5     void onStartAll(long[] nValues);
6     void onStopAll();
7     void onPredict(long predictN);
8     void onClear();
9     void onLogScaleChanged(boolean logScale);
10 }
```

Listing 6. Interfície ViewListener.

Tots els paràmetres són dades primitives o enums. No hi ha cap referència a *JButton*, *TextField* ni cap altre component visual. Aquesta és la clau de la separació MVC.

2) *ControlPanel*: Panell que conte els camps d'entrada, els botons d'execució i el camp de previsió. Les responsabilitats són exclusivament de la vista:

- 1) Capturar les accions de l'usuari (clics de botons, canvi de checkbox d'escala logaritmica).
- 2) Validar l'entrada internament: parsejar els camps de text a valors *long/int* i mostrar dialogs d'error (*JOptionPane*) si els valors són invalids.
- 3) Generar l'array de valors de  $n$  equidistants dins el rang  $[n_{inicial}, n_{final}]$  amb la fórmula:

$$n_i = n_{inici} + \frac{(n_{final} - n_{inici}) \cdot i}{p - 1}, \quad i = 0, \dots, p - 1 \quad (3)$$

on  $p$  és el nombre de passos. Només quan la validació passa satisfactòriament s'emet l'event semantic a través de *ViewListener*.

Quan un algorisme està en execució, el seu botó canvia d'icona (►→ ■) i de color (del color de l'algorisme a vermell), permetent aturar-lo individualment amb un segon clic.

3) *GraphPanel*: Panell personalitzat que sobreescriu *paintComponent(Graphics g)* per dibuixar la gràfica amb l'API *Graphics2D*. Les característiques principals són:

- **Eixos i graella**: eix X per a  $n$ , eix Y per al temps en mil·lisegons, amb marques numeriques i línies de graella de color gris clar.
- **Escala dual**: suporta escala lineal i logaritmica a l'eix Y, commutable en temps real. En escala logaritmica, la posició vertical es calcula com:

$$y = h - h \cdot \frac{\log_{10}(t) - \log_{\min}}{\log_{\max} - \log_{\min}} \quad (4)$$

- **Sèries de dades**: dibuixa punts (cercles plens de  $8 \times 8$  píxels) i línies de connexió per a cada algorisme amb el seu color assignat i un traçat de 2.5 punts.
- **Llegenda**: rectangle semitransparent amb fons blanc i opacitat 220/255 a la cantonada superior esquerra.
- **Formatejat dinamic**: les etiquetes s'adapten automàticament (ns,  $\mu$ s, ms, s, min per al temps; K, M per a  $n$ ).
- **Antialiasing**: activat tant per a línies com per a text per obtenir un renderitzat suau.

L'escala logaritmica és especialment útil quan es comparen algorismes amb ordres de magnitud molt diferents (per exemple,  $O(n)$  vs.  $O(n^3)$ ), ja que en escala logaritmica les funcions potencials  $n^k$  es visualitzen com rectes amb pendent  $k$ .

4) *ResultsPanel*: Panell dividit verticalment en dues parts mitjançant un *JSplitPane*:

- Una *JTable* no editable que mostra totes les mesures amb quatre columnes: algorisme,  $n$ , temps en ms i constant  $c$  en notació científica.
- Una *JTextArea* de només lectura amb font monoespaçada que mostra les constants multiplicatives mitjanes i les previsions sol·licitades per l'usuari, amb formatejat adaptatiu (mil·lisegons, segons, minuts, hores o dies).

5) *MainView*: *JFrame* principal que compon els tres subpanells en un *BorderLayout* (controls al NORTH, gràfica al CENTER, resultats a l'EAST). Es registra com a observadora del model amb una lambda:

```
1 model.addListener(() -> {
2     graphPanel.repaint();
3     resultsPanel.updateResults();
4 });
```

Listing 7. Registre com a observadora del model.

L'API pública de *MainView* consta exclusivament de mètodes que operen amb dades:

```
1 void addViewListener(ViewListener l);
2 void setAlgorithmRunning(
3     AlgorithmType t, boolean b);
4 void setLogScale(boolean logScale);
5 void showPredictions(long n,
```

```
Map<AlgorithmType, Double> preds);
```

Listing 8. API pública de MainView.

Cap mètode retorna ni rep un component Swing. Aquesta restricció és fonamental per a la separació MVC i ha estat un dels punts de disseny més importants del projecte.

#### D. Capa del Controlador

La classe Controller implementa ViewListener i gestiona el cicle de vida dels algorismes. El codi del mètode principal d'execució mostra el patró de SwingWorker:

```
1 private void startExecution(
2     AlgorithmType type, long[] nValues) {
3     SwingWorker<Void, Measurement> worker =
4         new SwingWorker<>() {
5         protected Void doInBackground() {
6             for (long n : nValues) {
7                 if (isCancelled()) break;
8                 // Escalfament JIT
9                 type.execute(Math.min(n, 500));
10                // Mesura real
11                long start = System.nanoTime();
12                type.execute(n);
13                long elapsed = System.nanoTime() - start;
14                publish(new Measurement(
15                    type, n, elapsed));
16            }
17            return null;
18        }
19        protected void process(
20            List<Measurement> chunks) {
21            for (Measurement m : chunks)
22                model.addMeasurement(m);
23        }
24        protected void done() {
25            workers.remove(type);
26            view.setAlgorithmRunning(type, false);
27        }
28    };
29    workers.put(type, worker);
30    view.setAlgorithmRunning(type, true);
31    worker.execute();
32 }
```

Listing 9. Execució d'un algorisme al controlador.

Cal destacar l'*escalfament* (warm-up) que es fa abans de cada mesura: s'executa l'algorisme amb una mida petita ( $\min(n, 500)$ ) per forçar la compilació JIT de la JVM. Això evita que la primera mesura estigui distorsionada pel cost de compilació.

El controlador manté un EnumMap de SwingWorker per poder cancel·lar cada algorisme individualment. **Mai no accedeix a cap component visual:** la seva única referència a la vista es a través dels mètodes de dades de MainView.

### V. CÀLCUL DEL COST ASIMPTÒTIC PER LA INSTRUCCIÓ CRÍTICA

La instrucció crítica de cada algorisme és l'operació dins el bucle més intern que s'executa el major nombre de vegades. El nombre total d'execucions d'aquesta instrucció determina  $f(n)$ .

#### A. Algorisme Lineal — $O(n)$

```
1 for (long i = 0; i < n; i++) {
2     sum += i * i; // instrucció crítica
3 }
```

Listing 10. Algorisme lineal.

La instrucció crítica  $\text{sum} += i * i$  s'executa exactament  $n$  vegades. No hi ha bucles imbricats, per tant:

$$f(n) = n \implies T(n) \in \Theta(n) \quad (5)$$

#### B. Algorisme Quasilineal — $O(n \cdot \log n)$

```
1 int logN = (int) (Math.log(n) / Math.log(2));
2 for (long i = 0; i < n; i++) {
3     for (int j = 0; j < logN; j++) {
4         sum += i ^ j; // instrucció crítica
5     }
6 }
```

Listing 11. Algorisme  $O(n \cdot \log n)$ .

El bucle extern itera  $n$  vegades i l'intern  $\lfloor \log_2 n \rfloor$  vegades per cada iteració externa. La instrucció crítica  $\text{sum} += i ^ j$  s'executa:

$$f(n) = n \cdot \lfloor \log_2 n \rfloor \implies T(n) \in \Theta(n \log n) \quad (6)$$

#### C. Algorisme Quadràtic — $O(n^2)$

```
1 for (long i = 0; i < n; i++) {
2     for (long j = 0; j < n; j++) {
3         sum += i * j; // instrucció crítica
4     }
5 }
```

Listing 12. Algorisme quadràtic.

Dos bucles imbricats, cadascun de  $n$  iteracions. La instrucció crítica  $\text{sum} += i * j$  s'executa:

$$f(n) = n \times n = n^2 \implies T(n) \in \Theta(n^2) \quad (7)$$

#### D. Algorisme Cúbic — $O(n^3)$

```
1 for (long i = 0; i < n; i++) {
2     for (long j = 0; j < n; j++) {
3         for (long k = 0; k < n; k++) {
4             sum += (i*j) + k; // inst. crítica
5         }
6     }
7 }
```

Listing 13. Algorisme cúbic.

Tres bucles imbricats, cadascun de  $n$  iteracions. La instrucció crítica  $\text{sum} += (i * j) + k$  s'executa:

$$f(n) = n^3 \implies T(n) \in \Theta(n^3) \quad (8)$$

#### E. Resum Comparatiu

La Taula II resumeix la instrucció crítica, el nombre d'execucions i la complexitat de cada algorisme.

Taula II  
INSTRUCCIÓ CRÍTICA I COMPLEXITAT DE CADA ALGORISME.

| Algorisme | Instrucció crítica | Execucions                   | $\Theta$   |
|-----------|--------------------|------------------------------|------------|
| Lineal    | sum += i * i       | $n$                          | $n$        |
| N Log N   | sum += i ^ j       | $n \lfloor \log_2 n \rfloor$ | $n \log n$ |
| Quadràtic | sum += i * j       | $n^2$                        | $n^2$      |
| Cúbic     | sum += (i * j) + k | $n^3$                        | $n^3$      |

### F. Constant Multiplicativa i Predicció

Per a cada mesura individual, la constant multiplicativa es calcula segons l'equació (2). La constant mitjana  $\bar{c}$  s'obté com la mitjana aritmètica de totes les constants individuals d'un algorisme:

$$\bar{c} = \frac{1}{m} \sum_{k=1}^m c_k = \frac{1}{m} \sum_{k=1}^m \frac{T(n_k)}{f(n_k)} \quad (9)$$

on  $m$  és el nombre de mesures realitzades. Aquesta constant reflecteix el cost temporal per unitat d'operació bàsica a la màquina concreta.

Per fer una predicció del temps d'execució per a un valor de  $n$  no mesurat, s'utilitza la fórmula:

$$\hat{T}(n) = \bar{c} \cdot f(n) \quad (10)$$

Per exemple, si tenim  $\bar{c} = 2.5$  ns per a l'algorisme quadràtic i volem predir  $n = 100\,000$ :

$$\hat{T}(100\,000) = 2.5 \times (100\,000)^2 = 2.5 \times 10^{10} \text{ ns} = 25 \text{ s} \quad (11)$$

Aquesta capacitat de predicció és un dels avantatges clau de l'anàlisi asimptòtic: amb unes poques mesures per a valors petits de  $n$ , es pot estimar el temps d'execució per a valors molt més grans sense necessitat d'executar l'algorisme.

## VI. RESULTATS I DISCUSSIÓ

L'aplicació implementada permet verificar experimentalment que els temps d'execució mesurats segueixen les funcions de cost teòric. A la gràfica, s'observa clarament com:

- La sèrie lineal  $O(n)$  creix de forma gairebé imperceptible en comparació amb les altres sèries.
- La sèrie  $O(n \cdot \log n)$  creix lleugerament més que la lineal, però de forma moderada i controlada.
- La sèrie quadràtica  $O(n^2)$  creix de forma parabòlica, essent significativament més costosa per a valors grans de  $n$ .
- La sèrie cúbica  $O(n^3)$  domina clarament totes les altres, fent-se impracticable per a valors de  $n$  moderadament grans.

L'ús de l'escala logarítmica a l'eix Y permet comparar totes quatre sèries simultàniament sense que les més ràpides quedin comprimides contra l'eix X. En escala logarítmica, les funcions potencials  $n^k$  es visualitzen com a rectes amb pendent proporcional a  $k$ , facilitant la verificació visual de la complexitat.

Les constants multiplicatives calculades es mantenen raonablement estables per a un algorisme donat, la qual cosa

confirma que el model asimptòtic  $T(n) = c \cdot f(n)$  és una bona aproximació. Petites variacions s'atribueixen a:

- Efectes de la jerarquia de memòria (cache L1/L2/L3 del processador): per a valors grans de  $n$ , les dades poden excedir la cache, augmentant la latència.
- La compilació JIT de la JVM: el compilador optimitza el codi progressivament durant l'execució, la qual cosa pot fer que les primeres mesures siguin més lentes. L'escalfament previ mitiga parcialment aquest efecte.
- La planificació del sistema operatiu: altres processos poden competir pel temps de CPU.

La funcionalitat de predicció ha mostrat resultats coherents: les prediccions per a valors de  $n$  dins el rang mesurat concorden amb les mesures reals, i les prediccions per a valors més grans segueixen l'extrapolació esperada.

Respecte al disseny del programari, l'estricta separació MVC amb comunicació per esdeveniments ha permès:

- Desenvolupar i provar la lògica del model independentment de la vista.
- Modificar la interfície gràfica (per exemple, afegir l'escala logarítmica o millorar el formatejat de la gràfica) sense alterar el controlador ni el model.
- Garantir que l'aplicació roman responsiva durant l'execució d'algorismes lents, gràcies a l'ús de `SwingWorker` que executa els algorismes en fils de fons separats del fil EDT.
- Cancel·lar algorismes individualment sense afectar els altres que s'estan executant concurrentment.

## VII. CONCLUSIONS

Aquesta pràctica ha permès aplicar de forma conjunta els conceptes d'anàlisi asimptòtic i de disseny de programari amb el patró MVC.

Des del punt de vista teòric, s'ha verificat experimentalment que els temps d'execució dels quatre algorismes implementats corresponen a les seves complexitats teòriques. La identificació de la instrucció crítica de cada algorisme —l'operació del bucle més intern— i el càlcul de la constant multiplicativa han permès fer prediccions fiables del temps d'execució per a mides d'entrada arbitràries. L'experiment confirma que el model  $T(n) = c \cdot f(n)$  és una aproximació vàlida quan  $n$  és prou gran i les condicions d'execució són estables.

Des del punt de vista del disseny de programari, s'ha aconseguit una implementació estricta del patró MVC on:

- 1) La vista **no exposa cap component visual** al controlador; tota interacció es fa a través d'esdeveniments semàntics.
- 2) La interfície `ViewListener` defineix set d'esdeveniments amb paràmetres de dades (enums, arrays de `long`, booleans), mai objectes `Swing`.
- 3) El model notifica canvis a través de `ModelListener` sense conèixer qui l'observa.
- 4) El controlador opera exclusivament amb dades, delegant la presentació visual a la vista i la lògica de negoci al model.
- 5) La concurrència es gestiona amb `SwingWorker`, garantint la responsivitat de la interfície i la cancel·lació segura.

El patró per esdeveniments ha demostrat ser essencial per mantenir el desacoblament entre capes, permetent que cada component evolucioni de forma independent. Aquesta separació no és una mera formalitat acadèmica: facilita el manteniment, la testabilitat i l'extensibilitat del programari. Per exemple, es podria substituir la vista Swing per una interfície JavaFX o web sense modificar ni una línia del model o del controlador.

Com a treball futur, es podrien afegir més algorismes (per exemple,  $O(2^n)$  o  $O(n!)$ ), implementar l'exportació de dades a CSV, permetre a l'usuari definir algorismes personalitzats, o afegir més estratègies de mesura com la repetició múltiple amb mitjana per reduir la variància experimental.

#### REFERÈNCIES

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [2] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, U.K.: Wiley, 1996.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [5] J. Elliott, R. Eckstein, M. Loy, D. Wood, and B. Cole, *Java Swing*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2002.
- [6] Oracle Corporation, "The Java Tutorials — Concurrency," [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- [7] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.